TD n° 5 et 6
Formes, héritage et polymorphisme
(2 séances)

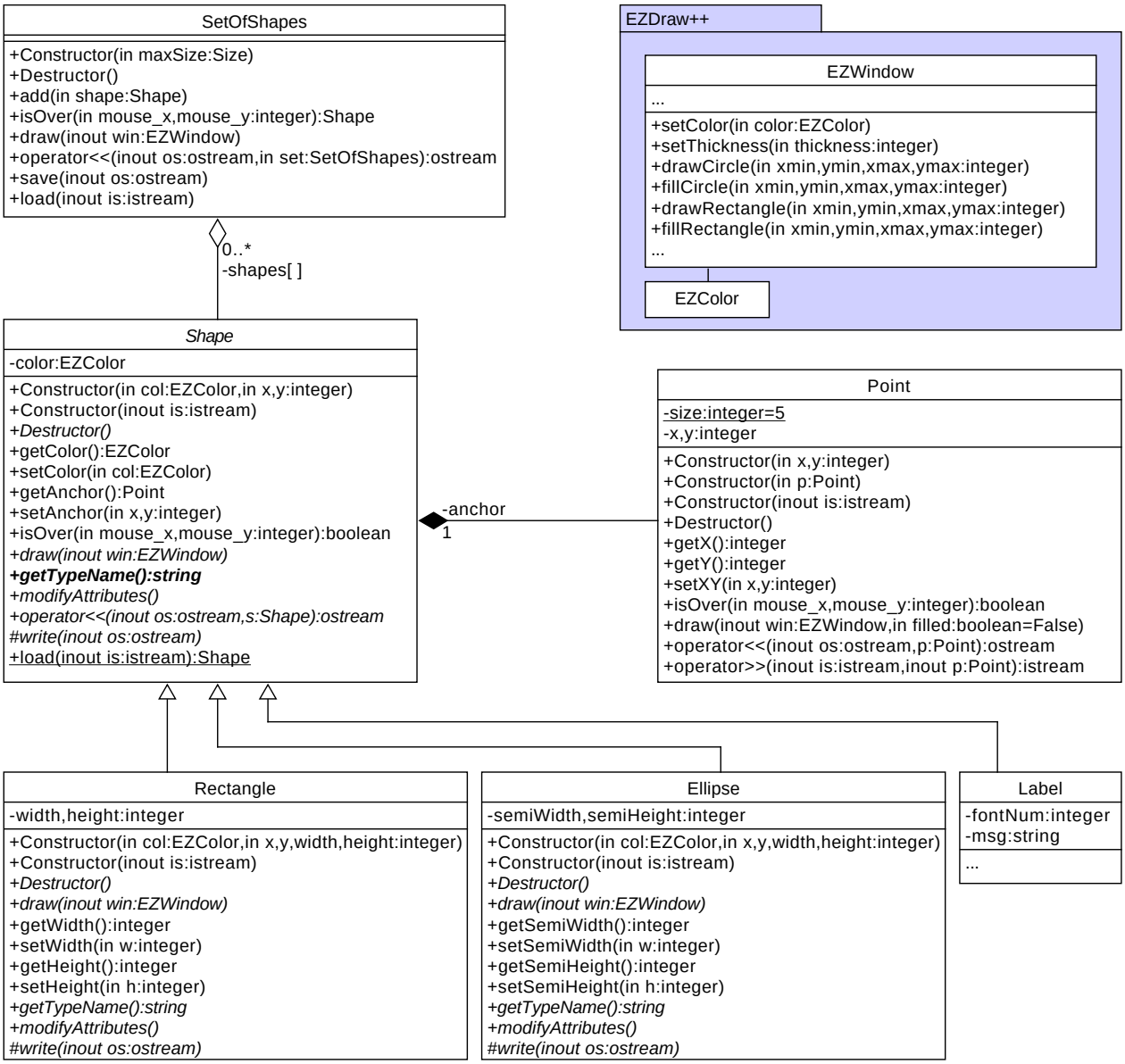
Eric REMY
eric.remy@univ-amu.fr

Version du 28 mars 2026

Cette planche d'exercices propose d'étudier l'implantation d'une série de classes dérivées de Shape permettant de représenter un ensemble de formes géométriques, puis de les tracer à l'écran ainsi que de les déplacer à la souris.

Exercice

Étudiez le diagramme UML ci-dessous et répondez aux questions.



La classe EZWindow et ses méthodes sont déjà implantées dans une mini-bibliothèque de classes fournie pour les TP et qui figure sur le diagramme par le package EZDraw++. Elles sont donc utilisables pour réaliser vos classes

dérivées de Shape. Dans les fonctions membres, (xmin, ymin) et (xmax, ymax) représentent respectivement les points en haut à gauche et en bas à droite de la boite englobante du dessin demandé.

1. Étudiez le diagramme de classe UML et vérifiez que vous comprenez toutes ses indications. Que signifie la flèche qui relie Rectangle et Shape ? Et celle qui relie Ellipse à Shape ? Quelle est la signification de la flèche qui relie Point et Shape ? Et celle qui relie SetOfShapes et Shape ? Comment les traduit-on en code C++ ? Si on voulait représenter la dépendance à la classe EZWindow, comment le ferait-on et pour quelle(s) classe(s) ?
2. Savez-vous pourquoi draw() est en italique dans les classes Shape, Rectangle et Ellipse ? Même question pour write() et getTypeName(). Pourquoi cette dernière fonction est-elle en gras ?
3. Pourquoi certaines données ou fonctions membres sont-elles soulignées ?
4. Puisque ce n'est pas indiqué dans le diagramme UML, d'après votre bon sens, quelles sont les méthodes que vous devrez marquer const, c'est-à-dire celles qui ne changent pas l'instance sur laquelle elles sont appelées ?
5. Dans EZDraw++, il est possible de tracer certaines formes de façon vide ou pleine. Où rajouteriez-vous l'accesseur' « bool ???:: isFilled () const » qui permet de tester si, par exemple, un rectangle ou une ellipse doit être tracé de façon pleine ? Même question pour « void ???:: setFilled (bool filled) » qui permet de régler cette propriété booléenne de remplissage. Même question pour le réglage de l'épaisseur du trait de dessin grâce aux fonctions « int ???:: getThickness() const » et « void ???:: setThickness(int thickness) ». Pensez à les ajouter sur le diagramme UML, ainsi que les attributs privés associés et les paramètres correspondants du constructeur.
6. Où rajouteriez-vous la classe Square et pourquoi ? Et pour la classe Circle ?
7. Donnez en C++ la déclaration et la définition des classes suivantes : Shape, Point, Rectangle, Square. Vous réaliserez seuls les classes Ellipse et Circle qui sont très similaires à Rectangle et Square. Pensez à marquer const les méthodes et les données qui peuvent l'être.
   Vous protégerez vos fichiers d'en-tête avec la technique classique, appelée en anglais « *include guard* » :

```
#ifndef MACLASSE_HPP // Si le symbole MACLASSE_HPP n'est pas défini
#define MACLASSE_HPP // le définir et intégrer le code fourni...

#include "ClasseParente.hpp"

class MaClasse : public ClasseParente {
...
};

#endif // ... jusqu'ici.
// Sinon il ne reste aucune ligne dans le fichier.
```

   Cela vous permettra d'inclure les déclarations dont dépend une classe dans le fichier d'en-tête de cette classe : en cas d'inclusions multiples, il n'y aura pas de redéfinition.
8. Réalisez la classe SetOfShapes qui permet de mémoriser une série d'au plus size formes quelconques. Implantez les méthodes suivantes : constructeur, destructeur, add(). Implanter draw() de façon à, pour chaque forme, la dessiner puis dessiner son ancre.
9. On veut que l'opération << appliquée à une Shape soit polymorphe, c'est-à-dire qu'elle affiche par exemple « `Rectangle #000000 10 20 50 60` » pour un rectangle noir dont l'ancre est en $(10, 20)$ et mesurant 50 pixels de large et 60 de haut ou encore « `Circle #FF0000 30 35 20` » pour un cercle rouge de 20 pixels de rayon centré en $(30, 35)$, suivant le véritable type de la Shape utilisée ? Or ce n'est pas une fonction membre, donc il n'est pas possible de la déclarer virtual. Voyez-vous une solution au problème ?
10. Grâce aux fonctions de Shape, créez une fonction save() membre de SetOfShapes qui sauvegarde dans un fichier de texte le nombre de formes puis chacune des figures actuellement présentes dans votre liste de formes.
11. Comment pourriez-vous réaliser dans Shape une fonction load() qui relit une des formes sauvegardées précédemment depuis le fichier sur le disque dur ? Indice : le premier mot rencontré à chaque fois donne à cette fonction le type de la forme à créer. Une fois que vous connaissez ce type, vous pouvez appeler dans *la bonne classe* une autre fonction load() qui va lire le nombre de paramètres dont elle *sait* avoir besoin et qui ensuite crée dynamiquement une nouvelle instance puis la renvoie. Ce type de fonction s'appelle une *factory* (une fonction qui fabrique des instances). Logiquement, cette fonction ne doit pas vous obliger à avoir déjà une instance pour fabriquer une instance (sinon le serpent se mord la queue), donc vous en ferez une fonction de classe (d'où le souligné dans le diagramme UML).
    Comment limiter le problème de l'interdépendance entre la classe mère (Shape) et les classes filles (Rectangle, Ellipse, Label, etc.) au moment de l'implantation ?
12. Vous pouvez maintenant réaliser une nouvelle fonction load() dans SetOfShapes dont le rôle sera de relire toutes les formes. Si vous avez bien compris, cette dernière fonction appelle celle de Shape qui appelle celle d'une des classes dérivées : en programmation orientée objet, cela s'appelle de la *délégation*. C'est aussi une technique très courante pour décomposer un problème de telle façon à ce que chaque niveau gère *sa propre* partie du problème.